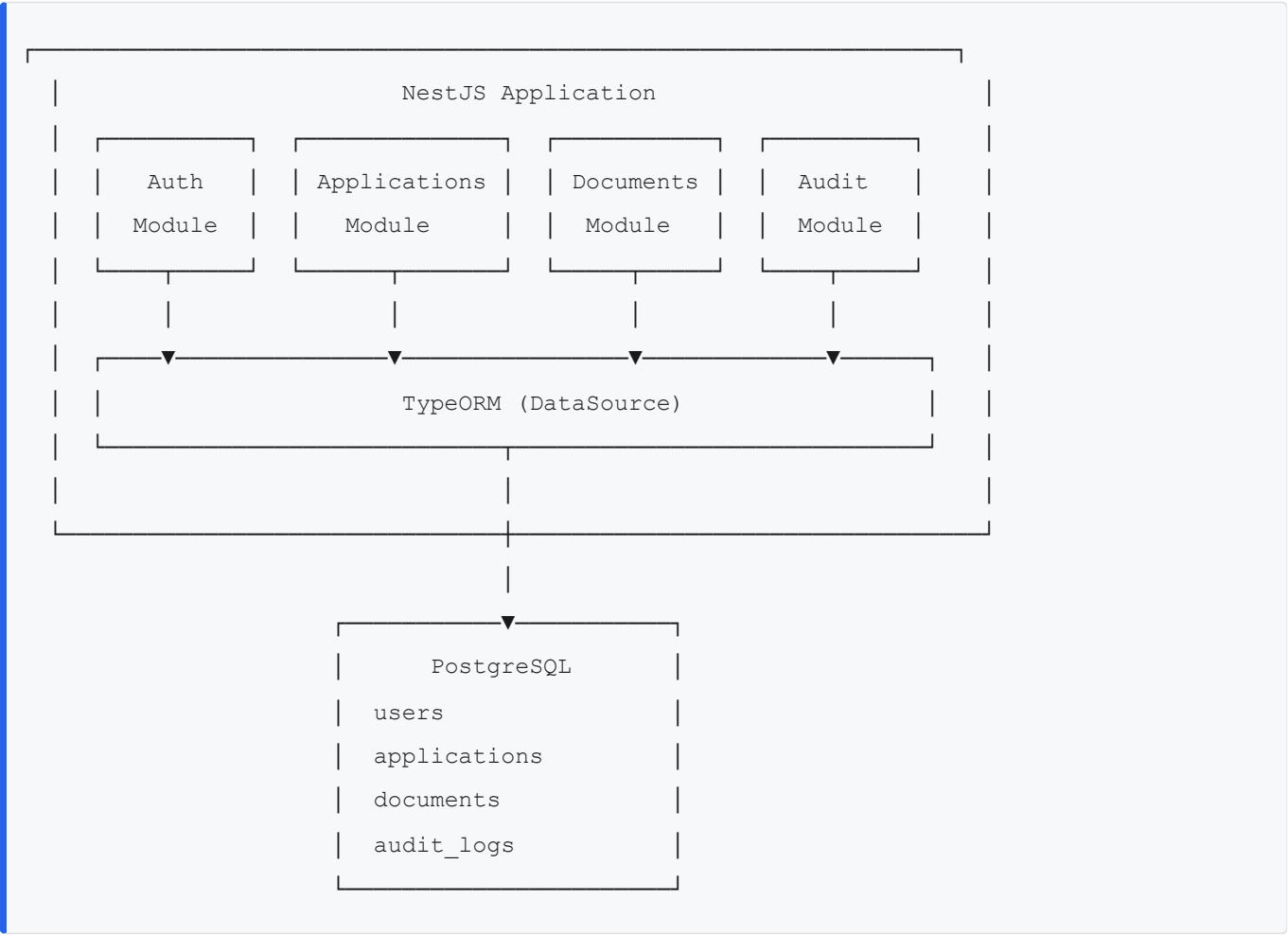


Bank Licensing & Compliance — Design Document

1. Architecture

Component Overview



Communication Pattern

All modules communicate through NestJS's dependency injection system. There is no message queue or inter-service HTTP.

Why a monolith, not microservices? Microservices would add distributed transaction complexity with no meaningful benefit at this scale.

2. Data Model

users

Column	Type	Notes
id	uuid PK	Generated
email	varchar	Unique
firstName	varchar	
lastName	varchar	
passwordHash	varchar	
role	enum	APPLICANT, REVIEWER, APPROVER, ADMIN
isActive	boolean	
createdAt	timestampz	
updatedAt	timestampz	

applications

Column	Type	Notes
id	uuid PK	
institutionName	varchar	
institutionType	varchar	
businessDescription	text	
registrationNumber	varchar	nullable
contactEmail	varchar	nullable
contactPhone	varchar	nullable
address	varchar	nullable
status	enum	See state below
applicantId	uuid FK	-> users.id
reviewerId	uuid FK	-> users.id, nullable, set on review start
approverId	uuid FK	-> users.id, nullable, set on decision
applicantComment	text	nullable, written by applicant
reviewNotes	text	nullable, written by reviewer
decisionNotes	text	nullable, written by approver
additionalInfoRequest	text	nullable, written by reviewer
version	integer	
createdAt	timestampz	
updatedAt	timestampz	

documents

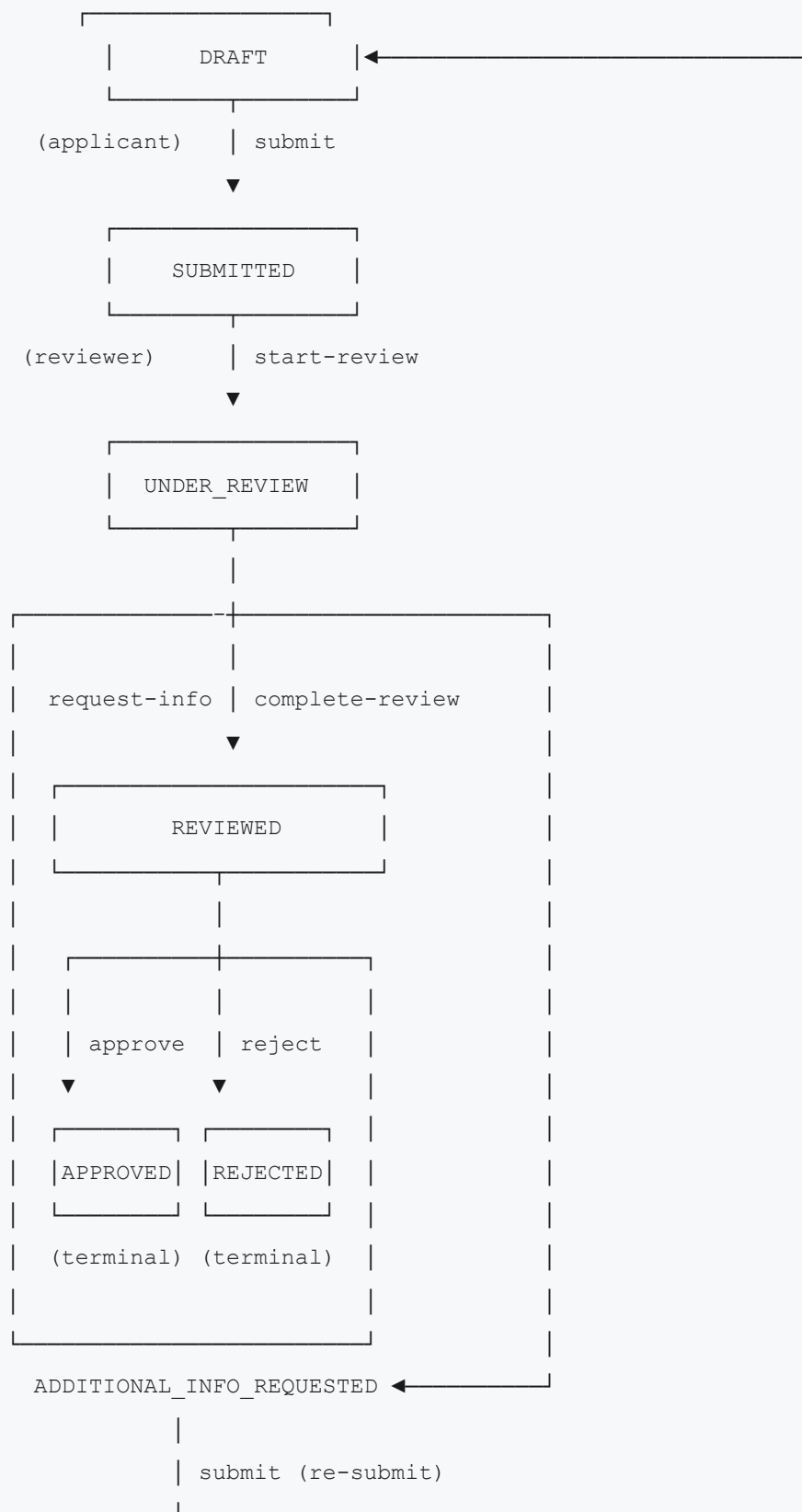
Column	Type	Notes
id	uuid PK	
originalName	varchar	Original filename from uploader
storedName	varchar	UUID-based name on disk
contentType	varchar	
fileSize	bigint	In bytes
applicationId	uuid FK	-> applications.id
uploaderId	uuid FK	-> users.id
documentGroup	uuid	
version	integer	Increments on re-upload of same document group
isSuperseded	boolean	True for all but the latest version of a group
uploadedAt	timestampz	

Document versioning: when an applicant re-uploads a document (e.g. after an info request), they supply the same `documentGroup` UUID. The service increments the version, marks all prior versions `isSuperseded = true`, and writes the new file. All versions remain retrievable.

audit_logs

Column	Type	Notes
id	uuid PK	
recordId	uuid	ID of the a resource
actingUserId	uuid	User ID performing the action
actingUserEmail	varchar	
actingUserRole	varchar	
action	enum	See AuditAction enum
stateBefore	jsonb	Full status snapshot before action
stateAfter	jsonb	Full status snapshot after action
metadata	jsonb	Action-specific extras (notes, fileSize etc)
ipAddress	varchar	
userAgent	text	
createdAt	timestampz	

3. State machine



State Definitions

State	Meaning
DRAFT	Created by applicant, not yet submitted. Editable.
SUBMITTED	Formally submitted, awaiting review assignment.
UNDER_REVIEW	A reviewer has taken ownership and is actively reviewing.
ADDITIONALINFOREQUESTED	Reviewer needs more information. Applicant can edit & resubmit.
REVIEWED	Reviewer has completed their analysis. Awaiting approval.
APPROVED	Terminal. Approver has granted the licence.
REJECTED	Terminal. Approver has denied the application.

Transition Rules

From	To	Actor	Constraints
DRAFT	SUBMITTED	APPLICANT	Must be the owning applicant
SUBMITTED	UNDER_REVIEW	REVIEWER	Any reviewer
UNDER_REVIEW	ADDITIONALINFOREQUESTED	REVIEWER	Must be assigned reviewer
UNDER_REVIEW	REVIEWED	REVIEWER	Must be assigned reviewer
ADDITIONALINFOREQUESTED	SUBMITTED	APPLICANT	Must be the owning applicant
REVIEWED	APPROVED	APPROVER	Approver ≠ Reviewer (hard rule)
REVIEWED	REJECTED	APPROVER	Approver ≠ Reviewer (hard rule)

Illegal transition enforcement: The `assertValidTransition()` method in `ApplicationsService` checks `VALID_TRANSITIONS[current]` and throws `BadRequestException` if the target state is not in the allowed list. This check runs inside the database transaction *after* acquiring a row lock, so it cannot be bypassed by concurrent requests.

4. Roles

Rationale for Role Design

The system models a real regulatory workflow. I identified three principal actors:

1. **The applicant** — an external institution applying for a licence.
2. **The reviewer** — an internal analyst who does due diligence.
3. **The approver** — a senior officer who makes the final decision.
4. **The admin** — manages the system and its users.

The key design constraint is **separation of duties** between reviewer and approver. This is a standard control in regulated environments: it prevents a single person from both recommending *and* approving, which would eliminate the second-pair-of-eyes check.

Role Permissions

Action	APPLICANT	REVIEWER	APPROVER	ADMIN
Create application	✓ (own)	✗	✗	✗
Edit application (DRAFT/INFO_REQUESTED)	✓ (own)	✗	✗	✗
Submit application	✓ (own)	✗	✗	✗
View own applications	✓	✗	✗	✗
View all applications	✗	✓	✓	✓
Upload documents	✓ (own)	✗	✗	✗
Download documents	✓ (own)	✓	✓	✓
Start review (SUBMITTED -> UNDER_REVIEW)	✗	✓	✗	✗
Request additional info	✗	✓	✗	✗
Complete review (-> REVIEWED)	✗	✓	✗	✗
Approve / Reject	✗	✗	✓	✗
View audit logs	✗	✓ (app)	✓	✓
View all audit logs	✗	✗	✓	✓
Create users	✗	✗	✗	✓
List users	✗	✗	✗	✓

5. Non-Negotiable Requirements: Implementation Detail

5.1 Authentication & Authorisation

Choice: JWT (stateless)

JWT was chosen over sessions for the following reasons:

- No server-side session store is needed, which simplifies deployment and horizontal scaling.
- JWTs are self-contained: the payload carries `userId`, `email`, and `role`.

- The `role` in the JWT is used for fast guard checks, but **every JWT-protected request re-fetches the user from the database** in `JwtStrategy.validate()`. This means a deactivated user (`isActive = false`) is denied immediately, even if their token hasn't expired.

Trade-off: JWTs cannot be revoked before expiry. We mitigate this by:

- Short token lifetime (8h, configurable via `JWT_EXPIRES_IN`).
- Re-validating `isActive` on every request from the database.
- Given more time: a token denylist in Redis would close this gap entirely.

Role enforcement in the backend: The `RolesGuard` reads the required roles from the `@Roles()` decorator and compares against `request.user.role`. This runs *after* the JWT guard loads the user from the database. A user who strips or modifies their JWT will fail JWT verification before reaching the roles check. A user who forges a valid JWT with a different role would need the signing secret, which is never exposed.

5.2 Workflow & State Integrity

Application State: Defined in `application.entity.ts` as `VALID_TRANSITIONS` map. This is the single source of truth, it's the same data structure used by the application service.

Illegal transitions at the API level: `assertValidTransition()` in `ApplicationsService` is called *inside* each database transaction. The HTTP layer cannot bypass it, even a direct database client calling a transition endpoint would hit this check.

Final states are permanent: `APPROVED` and `REJECTED` have empty transition arrays in `VALID_TRANSITIONS`. `assertValidTransition()` would throw for any attempted transition out of these states. No delete or status-reset endpoints exist.

Concurrent access: Two mechanisms work in concert:

1. **Pessimistic locking** (`SELECT ... FOR UPDATE`): All state transition methods acquire a row-level write lock at the start of their transaction. A second concurrent request on the same application will block until the first commits or rolls back. This is the primary defence.
2. **Optimistic locking** (`@VersionColumn`): TypeORM's version column provides a secondary check. If somehow two transactions read the same version and both attempt to write, the second write will fail with `OptimisticLockVersionMismatchError`, which the global exception filter converts to a `409 Conflict` response.

5.3 Audit Trail

Append-only enforcement is implemented at three levels:

1. **TypeORM lifecycle hooks:** `@BeforeUpdate()` and `@BeforeRemove()` on the `AuditLog` entity throw errors if TypeORM ever tries to update or delete a record through the ORM.
2. **No update/delete endpoints:** No controller or service method exposes a way to modify an audit log. There is no `DELETE /audit-logs/:id` or `PUT /audit-logs/:id`.

Legal evidence design:

- `actingUserEmail` and `actingUserRole` are snapshotted at write-time, not stored as foreign keys. If a user's email changes or their account is deleted, the audit record is unaffected.
- `ipAddress` and `userAgent` are recorded for non-repudiation.
- `stateBefore` and `stateAfter` capture the full relevant state as JSON, sufficient to reconstruct the history of any application.
- `createdAt` is set by `@CreateDateColumn()` (PostgreSQL `DEFAULT NOW()`), not passed from the application layer, preventing timestamp manipulation.

5.4 Document Handling

Simulated storage: Files are written to `./uploads/` (or a defined path in `.env`) on the local filesystem. The `Document` entity stores metadata (name, size, type, uploader, timestamp, path) in Postgres. In production, we can swap implementations and the write/read would go through the AWS SDK — the rest of the service is unchanged.

5MB enforcement: Enforced at two points:

1. Multer's `limits.fileSize` option rejects the multipart upload before it reaches the controller.
2. `DocumentsService.upload()` checks `file.size` again and throws `BadRequestException` if exceeded. This double-check ensures that any future bypass of the Multer middleware (e.g. a different interceptor, a test) still fails safely.

Versioning: The `documentGroup` UUID ties versions of the same logical document together. On re-upload with the same group, all prior rows are marked `isSuperseded = true` and a new row with an incremented `version` is inserted. All rows remain in the database and are returned by `GET /applications/:id/documents`, clearly labelled by `version` and `isSuperseded`.

5.5 API

Error contract:

- All errors go through `AllExceptionsFilter`, which catches everything including unhandled exceptions.
- Stack traces are logged internally (via `Logger`) but never returned in HTTP responses.
- `ForbiddenException` returns `403`. Unauthenticated requests return `401`. The JWT guard throws `401`, not `403`, so the distinction is correct.
- TypeORM's `OptimisticLockVersionMismatchError` is caught and converted to `409`.

Validation: `ValidationPipe` with `whitelist: true` strips any properties not declared in the DTO class. `forbidNonWhitelisted: true` returns a `400` if unknown properties are sent. This prevents parameter pollution attacks.

6. Trade-offs and What I Would Do Differently

Given more time

Postgres-level audit immutability: Add a migration that creates `RULE` or row-security policies preventing `UPDATE` / `DELETE` on `audit_logs`. The TypeORM hooks are a good application-layer guard but the database should be the last line.

Refresh tokens: The current JWT setup has no revocation mechanism short of waiting for expiry. A `refresh_tokens` table with short-lived access tokens (15 min) and longer-lived refresh tokens (7 days) would be more production-safe.

E2E tests: The unit tests cover the business rules thoroughly. Integration tests against a test database (using `@nestjs/testing` + a real Postgres in Docker) would give confidence that the ORM queries, transactions, and locks work as designed. I would use `testcontainers` to spin up a disposable Postgres instance.

Rate limiting: No rate limiting on the login endpoint. In production, `@nestjs/throttler` would be applied to prevent brute-force attacks.

File type validation: Currently, any MIME type is accepted. A production system should enforce an allowlist (PDF, DOCX, PNG, JPG) and validate the actual file bytes (magic numbers), not just the `Content-Type` header.

Notifications: When a reviewer requests additional info, the applicant gets no notification. In a real system, an email (via SES or similar) would be triggered from the service layer. I would add an event-driven notification module that listens to application status changes.

Cloud storage: Replace local filesystem writes with S3 presigned URLs or a similar object store, and store only the S3 key in the database.